

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 7A

Vistas Cruzadas

FASE 3 — Inscripciones

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear API Routes que cruzan datos de múltiples tablas y roles
- Implementar un endpoint que lista estudiantes inscritos en las materias de un docente
- Construir una vista admin de inscripciones con filtros compuestos (materia + estado)
- Usar el patrón de expansión: clic en una materia despliega sus estudiantes
- Implementar JOINS triples en Supabase (inscripciones → materias → estudiante)
- Activar el botón "Ver estudiantes" del docente (placeholder desde la Entrega 5C)
- Agregar políticas RLS para lectura cruzada entre roles
- Comprender cómo los 3 roles ven las mismas inscripciones desde perspectivas diferentes

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 6B completadas y funcionando
- ✓ Al menos 2 estudiantes con inscripciones (activas y retiradas)
- ✓ Al menos 2 materias del mismo docente con estudiantes inscritos
- ✓ Credenciales de los 3 roles: admin, docente, estudiante

☑ **Datos de prueba:** Para probar bien las vistas cruzadas, crea un segundo estudiante (desde admin o registro), inicia sesión con él e inscribelo en algunas materias. Así el docente verá múltiples estudiantes en sus materias.

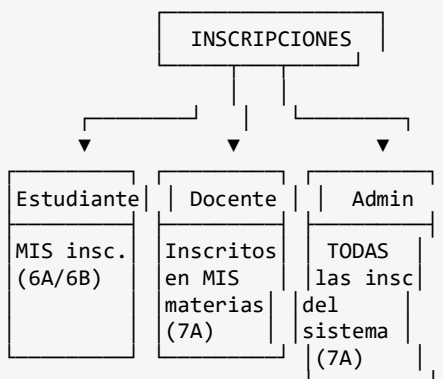
Teoría: Vistas Cruzadas entre Roles

¿Qué es una Vista Cruzada?

Hasta ahora, cada rol ve sus propios datos: el estudiante ve sus inscripciones, el docente ve sus materias. Una vista cruzada muestra datos que cruzan la frontera entre dos roles:

Vista cruzada	Quién ve	Qué datos cruza	Para qué sirve
Estudiantes por materia	Docente	Inscripciones → Estudiantes	Saber quiénes están en su clase
Todas las inscripciones	Admin	Inscripciones → Materias + Estudiantes	Visión completa del sistema

Diagrama: Perspectivas por rol sobre inscripciones



RLS para Vistas Cruzadas

El docente necesita leer inscripciones de sus materias, no solo las suyas propias. Esto requiere una política RLS que cruce las tablas:

Lógica RLS cruzada del docente

```
-- El docente puede ver inscripciones de SUS materias
-- Condición: la materia de la inscripción tiene docente_id = auth.uid()
```

```
SELECT inscripciones.*
FROM inscripciones
JOIN materias ON inscripciones.materia_id = materias.id
```

```
WHERE materias.docente_id = auth.uid() -- Solo materias del docente
```

i Subquery en RLS: Supabase permite usar subqueries en las políticas RLS. Usaremos EXISTS con una subconsulta para verificar que la materia pertenece al docente.

Paso 1 — Políticas RLS para Vistas Cruzadas

Antes de crear las API Routes, configuraremos las políticas RLS necesarias.

1.1 — Ejecutar en Supabase Dashboard

Ve a Supabase Dashboard → SQL Editor y ejecuta:

SQL — Políticas RLS para vistas cruzadas

```
-- 1. Docentes pueden ver inscripciones de SUS materias
CREATE POLICY "docentes_ver_inscripciones_sus_materias"
ON inscripciones
FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM materias
    WHERE materias.id = inscripciones.materia_id
    AND materias.docente_id = auth.uid()
  )
);

-- 2. Docentes pueden leer perfiles de estudiantes
-- (necesario para ver nombre/apellido de inscritos)
CREATE POLICY "docentes_ver_perfiles_estudiantes"
ON usuarios_perfil
FOR SELECT
USING (
  auth.uid() IS NOT NULL
  AND rol = 'estudiante'
);
```

1.2 — Explicación de las políticas

Política	Tabla	Quién	Condición
docentes_ver_inscripciones_sus_materias	inscripciones	Docente	La materia de la inscripción tiene docente_id = auth.uid()
docentes_ver_perfiles_estudiantes	usuarios_perfil	Cualquier autenticado	Solo perfiles con rol = 'estudiante'

☒ **EXISTS es eficiente:** PostgreSQL optimiza EXISTS con un índice en materias.docente_id. La subconsulta se detiene en cuanto encuentra una coincidencia, sin recorrer todas las filas.

i Admin usa Service Role: El admin NO necesita políticas RLS adicionales porque sus API Routes usan SERVICE_ROLE_KEY que ignora RLS completamente.

Paso 2 — API Route: Estudiantes Inscritos por Materia (Docente)

Este endpoint devuelve las materias del docente con sus estudiantes inscritos. El docente ve un resumen de cada materia y puede expandir para ver la lista de estudiantes.

2.1 — Crear la estructura

```
Terminal
mkdir -p app/api/docente/inscripciones
```

2.2 — Crear la API Route

Crea el archivo `app/api/docente/inscripciones/route.ts`:

```
app/api/docente/inscripciones/route.ts
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "docente") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // Obtener materias del docente
    const { data: materias, error: errMaterias } = await supabase
      .from("materias")
      .select("id, codigo, nombre, creditos, semestre")
      .eq("docente_id", user.id)
      .order("codigo", { ascending: true });

    if (errMaterias) {
```

```

    console.error("Error materias:", errMaterias);
    return NextResponse.json({ error: "Error al obtener materias" }, { status: 500 });
  }

  // Obtener inscripciones de esas materias
  const materiaIds = materias?.map((m) => m.id) || [];

  if (materiaIds.length === 0) {
    return NextResponse.json({ materias: [] });
  }

  const { data: inscripciones, error: errInsc } = await supabase
    .from("inscripciones")
    .select(`
      id, estado, created_at, materia_id,
      estudiante:usuarios_perfil!estudiante_id(id, nombre, apellido)
    `)
    .in("materia_id", materiaIds)
    .order("created_at", { ascending: true });

  if (errInsc) {
    console.error("Error inscripciones:", errInsc);
    return NextResponse.json({ error: "Error al obtener inscripciones" }, { status: 500 });
  }

  // Combinar: agregar inscripciones a cada materia
  const materiasConInscritos = materias?.map((m) => ({
    ...m,
    inscripciones: inscripciones?.filter((i) => i.materia_id === m.id) || [],
    totalInscritos: inscripciones?.filter((i) => i.materia_id === m.id && i.estado ===
"inscrito").length || 0,
  }));

  return NextResponse.json({ materias: materiasConInscritos });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
}
}

```

2.3 — Estructura de la respuesta

Ejemplo de respuesta JSON

```

{
  "materias": [
    {
      "id": "uuid-mat", "codigo": "MAT-101",
      "nombre": "Cálculo I", "creditos": 6, "semestre": "2026-1",
      "totalInscritos": 2,
      "inscripciones": [
        {
          "id": "uuid-insc-1", "estado": "inscrito",
          "estudiante": { "id": "uuid-est-1", "nombre": "Ana", "apellido": "Quispe" }
        },
        {
          "id": "uuid-insc-2", "estado": "retirado",
          "estudiante": { "id": "uuid-est-2", "nombre": "Carlos", "apellido": "Mamani" }
        }
      ]
    }
  ]
}

```

```
}  
  }  
    }  
  }  
}
```

☒ **Dos consultas, una respuesta:** Hacemos dos consultas (materias + inscripciones) y las combinamos en JS. Esto es más claro que un JOIN complejo y permite calcular totalInscritos por materia.

Paso 3 — API Route: Todas las Inscripciones (Admin)

El admin necesita ver todas las inscripciones del sistema con datos del estudiante, la materia y el docente.

3.1 — Crear la estructura

Terminal

```
mkdir -p app/api/admin/inscripciones
```

3.2 — Crear la API Route

Crea el archivo `app/api/admin/inscripciones/route.ts`:

app/api/admin/inscripciones/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    const { data: inscripciones, error } = await supabaseAdmin
      .from("inscripciones")
      .select(`
        id, estado, created_at,
```

```

        estudiante:usuarios_perfil!estudiante_id(id, nombre, apellido),
        materia:materias!materia_id(
            id, codigo, nombre, creditos, semestre,
            docente:usuarios_perfil!docente_id(nombre, apellido)
        )
    `)
    .order("created_at", { ascending: false });

    if (error) {
        console.error("Error al listar inscripciones:", error);
        return NextResponse.json({ error: "Error al obtener inscripciones" }, { status: 500 });
    }

    return NextResponse.json({ inscripciones });
} catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
}
}

```

i JOIN triple: Esta consulta cruza 3 tablas: inscripciones → estudiante (usuarios_perfil) + materia (materias) → docente (usuarios_perfil). Todo en un solo select().

Paso 4 — Componente: Estudiantes Inscritos (Docente)

Actualizaremos la vista del docente para mostrar sus materias con la lista de estudiantes inscritos en cada una, usando un patrón de expansión (clic para ver/ocultar).

4.1 — Actualizar MisMaterias

Reemplaza completamente components/dashboard/docente/mis-materias.tsx:

components/dashboard/docente/mis-materias.tsx – Parte 1/2

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { Button } from "@components/ui/button";
import { Badge } from "@components/ui/badge";
import {
  Card, CardContent, CardDescription,
  CardHeader, CardTitle,
} from "@components/ui/card";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import { RefreshCw, Users, ChevronDown, ChevronUp } from "lucide-react";

interface Estudiante {
  id: string;
  nombre: string;
  apellido: string;
}

interface Incripcion {
  id: string;
  estado: "inscrito" | "retirado";
  created_at: string;
  estudiante: Estudiante;
}

interface MateriaConInscritos {
  id: string;
  codigo: string;
  nombre: string;
  creditos: number;
  semestre: string;
  totalInscritos: number;
  inscripciones: Incripcion[];
}

export default function MisMaterias() {
  const [materias, setMaterias] = useState<MateriaConInscritos[]>([]);
  const [loading, setLoading] = useState(true);
  const [expandidas, setExpandidas] = useState<Set<string>>(new Set());

  const cargarDatos = async () => {
```

```

setLoading(true);
try {
  const res = await fetch("/api/docente/inscripciones");
  const data = await res.json();
  if (!res.ok) {
    toast.error(data.error || "Error al cargar datos");
    return;
  }
  setMaterias(data.materias);
} catch {
  toast.error("Error de conexión");
} finally {
  setLoading(false);
}
};

useEffect(() => { cargarDatos(); }, []);

const toggleExpandir = (materiaId: string) => {
  setExpandidas((prev) => {
    const next = new Set(prev);
    if (next.has(materiaId)) next.delete(materiaId);
    else next.add(materiaId);
    return next;
  });
};

```

☒ **Set para expandidos:** Usamos un `Set<string>` para rastrear qué materias están expandidas. Esto permite expandir múltiples materias simultáneamente.

4.2 — Renderizado con expansión

components/dashboard/docente/mis-materias.tsx — Parte 2/2

```

if (loading) {
  return (
    <div className="flex items-center justify-center py-12">
      <p className="text-muted-foreground">Cargando materias...</p>
    </div>
  );
}

if (materias.length === 0) {
  return (
    <div className="flex flex-col items-center justify-center py-12 gap-2">
      <p className="text-muted-foreground">No tienes materias asignadas.</p>
    </div>
  );
}

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <p className="text-sm text-muted-foreground">
        {materias.length} materia(s) |{" "}
        {materias.reduce((s, m) => s + m.totalInscritos, 0)} estudiante(s) inscrito(s)
      </p>
      <Button variant="outline" size="sm" onClick={cargarDatos}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>
    </div>

    {/* Cards de materias con expansión */}
    <div className="space-y-3">
      {materias.map((m) => {
        const isExpanded = expandidas.has(m.id);
        return (
          <Card key={m.id}>
            <CardHeader
              className="cursor-pointer hover:bg-muted/50 transition-colors"
              onClick={() => toggleExpandir(m.id)}
            >
              <div className="flex items-center justify-between">
                <div>
                  <CardTitle className="text-base flex items-center gap-2">
                    <Badge variant="outline" className="font-mono">{m.codigo}</Badge>
                    {m.nombre}
                  </CardTitle>
                  <CardDescription>
                    {m.creditos} créd. | {m.semestre}
                  </CardDescription>
                </div>
                <div className="flex items-center gap-3">
                  <Badge variant={m.totalInscritos > 0 ? "default" : "secondary"}>
                    <Users className="mr-1 h-3 w-3" />
                    {m.totalInscritos} inscrito(s)
                  </Badge>
                  {isExpanded
                    ? <ChevronUp className="h-5 w-5 text-muted-foreground" />
                    : <ChevronDown className="h-5 w-5 text-muted-foreground" />
                  }
                </div>
              </div>
            </CardHeader>

```

```

    {isExpanded && (
      <CardContent>
        {m.inscripciones.length === 0 ? (
          <p className="text-sm text-muted-foreground text-center py-4">
            Ningún estudiante inscrito aún.
          </p>
        ) : (
          <div className="rounded-md border">
            <Table>
              <TableHeader>
                <TableRow>
                  <TableHead>Estudiante</TableHead>
                  <TableHead>Estado</TableHead>
                  <TableHead>Fecha inscripción</TableHead>
                </TableRow>
              </TableHeader>
              <TableBody>
                {m.inscripciones.map((i) => (
                  <TableRow key={i.id}>
                    <TableCell className="font-medium">
                      {i.estudiante?.nombre} {i.estudiante?.apellido}
                    </TableCell>
                    <TableCell>
                      <Badge
                        variant={i.estado === "inscrito" ? "default" : "destructive"}
                      >
                        {i.estado.charAt(0).toUpperCase() + i.estado.slice(1)}
                      </Badge>
                    </TableCell>
                    <TableCell>
                      {new Date(i.created_at).toLocaleDateString("es-BO")}
                    </TableCell>
                  </TableRow>
                ))}
              </TableBody>
            </Table>
          </div>
        )}
      </CardContent>
    )}
  </Card>
);
}}
</div>
</div>
);
}

```

i La página no cambia: `app/(dashboard)/docente/materias/page.tsx` sigue importando `MisMaterias`. No necesita modificaciones porque el componente se actualizó in situ.

Paso 5 — Componente: Inscripciones del Sistema (Admin)

El admin ve una tabla con todas las inscripciones, con filtros por materia y estado.

5.1 — Crear el componente

Crea el archivo `components/dashboard/admin/inscripciones-table.tsx`:

components/dashboard/admin/inscripciones-table.tsx — Parte 1/2

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { RefreshCw } from "lucide-react";

interface Inscripcion {
  id: string;
  estado: "inscrito" | "retirado";
  created_at: string;
  estudiante: { id: string; nombre: string; apellido: string };
  materia: {
    id: string; codigo: string; nombre: string;
    creditos: number; semestre: string;
    docente: { nombre: string; apellido: string } | null;
  };
}

export default function InscripcionesTable() {
  const [inscripciones, setInscripciones] = useState<Inscripcion[]>([]);
  const [loading, setLoading] = useState(true);
  const [filtroMateria, setFiltroMateria] = useState<string>("todos");
  const [filtroEstado, setFiltroEstado] = useState<string>("todos");

  const cargarInscripciones = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/admin/inscripciones");
      const data = await res.json();
      if (!res.ok) {
        toast.error(data.error || "Error al cargar inscripciones");
        return;
      }
      setInscripciones(data.inscripciones);
    } catch {
      toast.error("Error de conexión");
    } finally {

```

```

        setLoading(false);
    }
};

useEffect(() => { cargarInscripciones(); }, []);

// Materias únicas para filtro
const materiasUnicas = [
    ...new Map(
        inscripciones.map((i) => [i.materia?.id, i.materia])
    ).values(),
];

const inscripcionesFiltradas = inscripciones.filter((i) => {
    const cumpleMateria = filtroMateria === "todos" || i.materia?.id === filtroMateria;
    const cumpleEstado = filtroEstado === "todos" || i.estado === filtroEstado;
    return cumpleMateria && cumpleEstado;
});

```

☒ **Map para deduplicar:** Usamos new Map() con el ID como clave para obtener materias únicas del array de inscripciones. Esto genera el Select de filtro dinámicamente.

5.2 — Renderizado de la tabla

components/dashboard/admin/inscripciones-table.tsx — Parte 2/2

```
return (
  <div className="space-y-4">
    { /* Filtros */ }
    <div className="flex items-center gap-4 flex-wrap">
      <Select value={filtroMateria} onChange={setFiltroMateria}>
        <SelectTrigger className="w-[220px]">
          <SelectValue placeholder="Filtrar por materia" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todas las materias</SelectItem>
          {materiasUnicas.map((m) => m && (
            <SelectItem key={m.id} value={m.id}>
              {m.codigo} - {m.nombre}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>

      <Select value={filtroEstado} onChange={setFiltroEstado}>
        <SelectTrigger className="w-[180px]">
          <SelectValue placeholder="Filtrar por estado" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los estados</SelectItem>
          <SelectItem value="inscrito">Inscritos</SelectItem>
          <SelectItem value="retirado">Retirados</SelectItem>
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarInscripciones}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>

      <span className="text-sm text-muted-foreground ml-auto">
        {inscripcionesFiltradas.length} inscripción(es)
      </span>
    </div>

    { /* Tabla */ }
    <div className="rounded-md border">
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Estudiante</TableHead>
            <TableHead>Materia</TableHead>
            <TableHead>Créditos</TableHead>
            <TableHead>Semestre</TableHead>
            <TableHead>Docente</TableHead>
            <TableHead>Estado</TableHead>
            <TableHead>Fecha</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {loading ? (
            <TableRow>
              <TableCell colspan={7} className="text-center py-8">
                Cargando inscripciones...
              </TableCell>
            </TableRow>
          )
        </TableBody>
      </Table>
    </div>
  )
}
```

```

) : inscripcionesFiltradas.length === 0 ? (
  <TableRow>
    <TableCell colSpan={7} className="text-center py-8">
      No se encontraron inscripciones
    </TableCell>
  </TableRow>
) : (
  inscripcionesFiltradas.map((i) => (
    <TableRow key={i.id}>
      <TableCell className="font-medium">
        {i.estudiante?.nombre} {i.estudiante?.apellido}
      </TableCell>
      <TableCell>
        <Badge variant="outline" className="font-mono">
          {i.materia?.codigo}
        </Badge>
        {" "}{i.materia?.nombre}
      </TableCell>
      <TableCell>
        <Badge variant="secondary">{i.materia?.creditos}</Badge>
      </TableCell>
      <TableCell>{i.materia?.semestre}</TableCell>
      <TableCell>
        {i.materia?.docente
          ? `${i.materia.docente.nombre} ${i.materia.docente.apellido}`
          : <span className="text-muted-foreground italic">Sin asignar</span>}
      </TableCell>
      <TableCell>
        <Badge
          variant={i.estado === "inscrito" ? "default" : "destructive"}
        >
          {i.estado.charAt(0).toUpperCase() + i.estado.slice(1)}
        </Badge>
      </TableCell>
      <TableCell>
        {new Date(i.created_at).toLocaleDateString("es-B0")}
      </TableCell>
    </TableRow>
  ))
)}
</TableBody>
</Table>
</div>
</div>
);
}

```

Paso 6 — Página Admin: Inscripciones

Reemplaza el placeholder de admin/inscripciones con la página real.

app/(dashboard)/admin/inscripciones/page.tsx

```
import {
  Card, CardContent, CardDescription,
  CardHeader, CardTitle,
} from "@components/ui/card";
import { ClipboardList } from "lucide-react";
import InscripcionesTable from "@components/dashboard/admin/inscripciones-table";

export default function AdminInscripcionesPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div className="flex items-center gap-3">
        <ClipboardList className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Inscripciones del Sistema
          </h1>
          <p className="text-muted-foreground">
            Visión completa de todas las inscripciones registradas
          </p>
        </div>
      </div>
      <div>
        <Card>
          <CardHeader>
            <CardTitle>Registro de Inscripciones</CardTitle>
            <CardDescription>
              Todas las inscripciones con datos de estudiante, materia y docente
            </CardDescription>
          </CardHeader>
          <CardContent>
            <InscripcionesTable />
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
Supabase: 2 políticas RLS	Nuevo (SQL)	Docente ve inscripciones de sus materias + perfiles estudiantes
app/api/docente/inscripciones/routes.ts	Nuevo	GET: materias del docente con lista de inscritos
app/api/admin/inscripciones/route.ts	Nuevo	GET: todas las inscripciones con JOIN triple
components/dashboard/docente/mis-materias.tsx	Modificado (reemplazo)	Cards expandibles con tabla de estudiantes por materia
components/dashboard/admin/inscripciones-table.tsx	Nuevo	Tabla completa con filtros por materia y estado
app/(dashboard)/admin/inscripciones/page.tsx	Modificado	Reemplaza placeholder con InscripcionesTable

Verificación Final

Ejecuta `npm run dev` y prueba con los 3 roles:

A. Vista Docente — Mis Materias (actualizada)

- ✓ Iniciar sesión como docente → `/docente/materias`
- ✓ Cada materia se muestra como Card con Badge de inscritos
- ✓ El contador global muestra total de materias y estudiantes
- ✓ Clic en una Card la expande mostrando la tabla de estudiantes
- ✓ El icono `ChevronDown` cambia a `ChevronUp` al expandir
- ✓ La tabla muestra nombre, estado (Badge) y fecha de inscripción
- ✓ Estudiantes inscritos tienen Badge azul, retirados Badge rojo
- ✓ Se pueden expandir múltiples materias simultáneamente
- ✓ Materias sin inscritos muestran mensaje "Ningún estudiante inscrito"

B. Vista Admin — Inscripciones del Sistema

- ✓ Iniciar sesión como admin → `/admin/inscripciones`
- ✓ La tabla muestra 7 columnas: estudiante, materia, créditos, semestre, docente, estado, fecha
- ✓ Filtro por materia muestra la lista de materias con código y nombre
- ✓ Filtro por estado funciona (inscritos / retirados / todos)
- ✓ Combinar ambos filtros funciona correctamente
- ✓ El contador muestra el número correcto de resultados filtrados

C. Vistas anteriores siguen funcionando

- ✓ Estudiante: catálogo con botones de inscripción funciona
- ✓ Estudiante: mis inscripciones con retiro funciona
- ✓ Admin: gestión de usuarios funciona
- ✓ Admin: gestión de materias (CRUD) funciona

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Estadísticas del docente: En la parte superior de MisMaterias, agrega 3 StatsCard: Total de materias asignadas, Total de estudiantes inscritos (activos) y Promedio de estudiantes por materia. Los datos se calculan de las materias cargadas.
2. Exportar lista del admin: Agrega un botón "Exportar CSV" en la tabla de admin que genere un archivo CSV con las inscripciones filtradas. Usa Blob + URL.createObjectURL + enlace temporal para descargar. Incluye columnas: estudiante, materia, créditos, estado, fecha.
3. Búsqueda por estudiante: En la tabla del admin, agrega un Input de búsqueda que filtre por nombre o apellido del estudiante. Combina con los filtros existentes de materia y estado.
4. Detalle del estudiante para el docente: Al hacer clic en el nombre de un estudiante en la tabla expandida del docente, muestra un Dialog con información del estudiante: nombre completo, fecha de inscripción, estado, y una lista de todas las materias en las que está inscrito (no solo las del docente actual).

i Para entregar: Implementa al menos 2 de los 4 ejercicios. Toma capturas de pantalla mostrando: la vista del docente expandida con estudiantes, y la tabla del admin con filtros aplicados.

Próximo: Entrega 7B — Dashboards y Cierre MVP

En la última entrega del proyecto completaremos el MVP:

- Dashboard del estudiante: materias inscritas, créditos, estadísticas
- Dashboard del docente: resumen de materias, total de estudiantes
- Dashboard del admin: estadísticas globales del sistema
- Componentes StatsCard con datos reales calculados desde las APIs
- Checklist final de cierre del MVP completo

☑ **Preparación:** Asegúrate de tener datos variados en el sistema: múltiples estudiantes con inscripciones mixtas, varias materias con y sin docente, usuarios activos e inactivos. Estos datos nutrirán los dashboards.

Fin de la Entrega 7A